Guía de Estudio: Backend, Bases de Datos, React y Arquitectura Cloud

Backend and APIs

• HTTP: Métodos GET, POST, PUT, PATCH y DELETE. Comprender códigos 2xx, 4xx y 5xx, encabezados HTTP y principios REST como recursos, stateless y versionado de APIs.

• GraphQL: esquemas, tipos, queries, mutations, resolvers y manejo de errores. Diferencias frente a REST.

• Autenticación vs autorización. JWT, OAuth, sesiones y validación de permisos en el servidor.

• Separación de controladores, lógica de negocio y capa de persistencia.

• Procesamiento asíncrono, colas, reintentos, timeouts, idempotencia y manejo de eventos duplicados.

Preguntas de repaso:

1. Explica este tema con tus palabras. 2. ¿Cuáles son los errores más comunes? 3. ¿Cómo se aplica en un proyecto real?

Databases and ORMs

• Modelado relacional, claves primarias y foráneas, relaciones 1:1, 1:N y N:M.

• Normalización y restricciones de integridad.

• Consultas SQL con WHERE, JOIN, GROUP BY, índices y planes de ejecución.

• Transacciones y propiedades ACID.

• Migraciones seguras y cambios compatibles hacia atrás.

• Ventajas y limitaciones de ORM. Revisar SQL generado cuando exista bajo rendimiento.

• Paginación, problema N+1, bloqueos y diagnóstico de rendimiento.

Preguntas de repaso:

1. Explica este tema con tus palabras. 2. ¿Cuáles son los errores más comunes? 3. ¿Cómo se aplica en un proyecto real?

Frontend y React

• Componentes, props, estado y composición.

• Hooks principales: useState, useEffect, useMemo y useCallback.

• Evitar estado innecesario y closures obsoletos.

• Manejo de formularios, validaciones y mensajes de usuario.

• Estados de carga, error y ausencia de datos.

• Routing, vistas protegidas y consumo de APIs.

• Caché e invalidación de datos.

• Accesibilidad, diseño responsive y renderizado seguro de contenido.

Preguntas de repaso:

1. Explica este tema con tus palabras. 2. ¿Cuáles son los errores más comunes? 3. ¿Cómo se aplica en un proyecto real?

Arquitectura y Sistemas Cloud

• Monolito, monolito modular, servicios y microservicios: ventajas y desventajas.

• Límites de servicio y dirección de dependencias.

• Arquitecturas orientadas a eventos y consistencia eventual.

• Colas de mensajes y aislamiento de fallos.

• Conceptos serverless, gestión de secretos, despliegue y observabilidad.

• Resiliencia: timeouts, reintentos con backoff, idempotencia y degradación elegante.

Preguntas de repaso:

1. Explica este tema con tus palabras. 2. ¿Cuáles son los errores más comunes? 3. ¿Cómo se aplica en un proyecto real?

Resumen para entrevistas técnicas

Practica explicaciones claras, dibuja arquitecturas simples, justifica decisiones técnicas y compara alternativas. Prioriza comprensión de fundamentos sobre memorización.